AIMS System - Detailed Class Design Documentation

1. Design Class: Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productID	int	null	Unique identifier for the product
2	title	String	null	Name of the product
3	category	String	null	Product category (Book, CD, DVD, or LP record)
4	value	float	0.0	Base price of the product before VAT
5	price	float	0.0	Selling price of the product
6	quantity	int	0	Available quantity in stock
7	description	String	null	Product description
8	imageURL	String	null	The URL for the image of the product item
9	barCode	String	null	Unique barcode for the product
10	warehouseEntryDate	Date	null	Date when the product entered the warehouse
11	dimensions	String	null	Physical dimensions of the product (e.g., 10x20x5 cm)
12	weight	float	null	Weight of the product in kilograms
13	warehouseProvince	String	null	province of warehouse storing the product
14	warehouseDistrict	String	null	district of warehouse storing the product
15	warehouseAddress	String	null	detailed address of warehouse storing the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	checkProductAvailability	boolean	Checks if the product has sufficient quantity available
2	updateProduct	void	Updates product information in the system
3	checkEligibilityOfRushDelivery	boolean	Checks if product is eligible for rush delivery
4	createProduct	void	Create a new product in the system
5	searchProduct	void	Search for a specific product attributes

Parameter:

- checkProductAvailability(quantity: int): Boolean
    - **quantity**: An integer representing the number of products to check if the available inventory is sufficient.
    - **Return:** true if the available inventory is sufficient, otherwise false.
- updateProduct(productID: int): void
    - **productID:** productID must exist
- searchProduct(searchCriteria: Map<String, Object>): void
    - **searchCriteria:** A map containing search criteria (field names and values)

Exception:

- updateProduct(productID: int): void
    - ProductNotFoundException: If no product with the specified productID is found.
    - ExceedUpdateTimesException: If update more than 30 products a day
    - ExceedUpdatePriceTimesPerDayException: If update price more than twice a day

Method:

How to use parameters/attributes:

- checkProductAvailability(quantity: int): Boolean
    - **Purpose**: This method checks whether the available inventory is sufficient for the requested quantity of a product.
    - **Usage**: If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.
- updateProduct(productID: int): void
    - **Purpose**: This method updates the details of an existing product based on the provided information.
    - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- createProduct(): void
    - **Purpose**: This method creates a new product with the provided information.
    - **Usage:** If all required attributes are provided and valid, the product is successfully created and added to the inventory.
- checkEligibilityOfRushDelivery(address: String): Boolean
    - **Purpose**: to check rush delivery support
    - **Usage**: compare ‘address’ parameter (from delivery information) and address of product (based on information provided by Product Manager)
- searchProduct(searchCriteria: Map<String, Object>): void
    - **Purpose**: Searches for products matching specific criteria.
    - **Usage**: Used to find products based on various attributes.
    - **Processing**:
        1. Constructs a database query based on the search criteria
        2. Executes the query
        3. Returns matching products

State:

- If stock > requested quantity → Available.
- If stock < requested quantity → Not Available.
- If product_id is invalid → ProductNotFoundException is raised.

2. Design Class: PaymentTransaction

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	int	null	Unique identifier for the transaction
2	amount	float	0.0	Payment amount
3	paymentMethod	String	"Credit Card"	Method of payment
4	transactionDate	Date	current date	Date of transaction
5	status	String	"Pending"	Transaction status (Pending, Success, Failed, Refunded)
6	content	String	null	Transaction details or description
7	orderId	int	null	ID of the associated order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	evalutePaymentResult	boolean	Processes the payment result from VNPay
2	save	void	Saves the transaction to the database
3	transactionInfo	String	Returns formatted transaction information
4	saveRefundTransaction	void	Saves refund transaction information
5	createTransaction	void	Creates (stores) a new payment transaction in the system

Parameter:

- transactionInfo(transactionId: String) : PaymentTransaction
    - transactionId: An integer representing the unique ID of the transaction.
    - Return: A PaymentTransaction object containing the transaction details.

Exception:

- TransactionNotFoundException if referencing a non-existent transaction.
- InvalidTransactionAmountException – If the amount is invalid (e.g., negative or zero when it shouldn't be).

Method:

- save()
    - Purpose: Commits the current transaction state to persistent storage.
    - Usage: Called after creating or updating any transaction details (e.g., setting the status to "completed").
- transactionInfo(transactionId: String): PaymentTransaction
    - Purpose: Retrieves the details of a specific payment transaction by its unique ID.
    - Usage: If transactionId exists, returns a PaymentTransaction object. If it does not exist, raises TransactionNotFoundException.

State:

- If transactionId is invalid → TransactionNotFoundException is raised.
- If transactionStatus is "completed" → Payment is successful and no further changes unless a refund is requested.
- If processRefund is successful → transactionStatus might be changed to "refunded", and isRefunded is set to true.
- refundTransactionId may be generated or updated when a refund is initiated.

3. Design Class: OrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentOrder	Order	null	Currently selected order
2	orderList	List<Order>	empty	A list of orders loaded for viewing or processing
3	alreadyProcessed	boolean	false	Indicates if an order has already been processed or canceled

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	approveOrder	void	Approves a pending order
2	rejectOrder	void	Rejects a pending order with reason
3	viewOrderList	void	Retrieves or displays a list of pending orders.
4	getOrderDetails	void	Gets details for a specific order
5	checkAvailableInventory	boolean	Checks if there's enough inventory for an order
6	processCancel	boolean	Processes the cancellation of an order
7	createOrder	Order	Creates a new order in the system
8	refundConfirm	void	Initiates or confirms a refund operation via VNPay.
9	selectCancel	boolean	Handles user requests to cancel the order.
10	checkAlreadyProcessed	boolean	Checks if the current order is already processed (approved/rejected).
11	getOrder	Order	Returns the current Order object in the controller

Parameter.

- checkAlreadyProcessed(orderId: String): boolean
    - orderId: The ID of the order to check
    - Return: true if the order is already approved or rejected, otherwise false.
- selectCancel(orderId: String): boolean
    - orderId: The order to attempt to cancel.
    - Return: true if cancellation is selected, false otherwise.
- refundConfirm(orderId: String): void
    - orderId: The order to be refunded.
    - processCancel(orderId: String): boolean
- ApproveOrder(orderID: String): void
    - orderID: The ID of the order to approve
- RejectOrder(orderID: String): void
    - orderID: The ID of the order to reject
- processCancel(orderId: String): boolean
    - orderId: The ID of the order to finalize cancellation for.

    - Return: true if successfully canceled, otherwise false.
- GetOrderDetail(orderId: String)
    - orderId: The ID of the order whose details are to be fetched.
    - Return: detail information

Exception

- OrderNotFoundException: If the specified orderId does not exist.
- OrderAlreadyApprovedException: If trying to approve/reject/cancel an order that is already processed.

Method

- approveOrder(orderId)
    - Purpose: Set order status to "approved" and decrement inventory.
    - Usage: Called by product manager or automated system upon reviewing an order.
- rejectOrder(orderId)
    - Purpose: Set order status to "rejected". Possibly triggers a refund if payment was collected.
- checkAlreadyProcessed(orderId)
    - Purpose: Quickly verify whether an order is no longer in "pending" state.
- selectCancel(orderId) and processCancel(orderId)
    - Purpose: used in two-step cancellation processes.

State

- If orderStatus is approved or rejected → The order is processed, thus cannot be changed.
- If product is insufficient → approveOrder might either reject the order or throw an exception.

4. Design Class: Order

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	Int	"Pending"	Unique identifier for the order
2	status	String	empty list	Current status: "pending," "approved," "rejected," "canceled," etc.
3	orderItems	List<OrderItem>	empty list	Collection of items (products + quantity) in this order
4	rushOrderItems	List<OrderItem>	empty list	Collection of items eligible for rush delivery
5	regularOrderItems	List<OrderItem>	empty list	Collection of items for regular delivery
6	createdDate	DateTime	current time	Timestamp when the order is first created
7	subtotal	float	0.0	Sum of all product prices excluding VAT
8	vatAmount	float	0.0	Calculated VAT amount (10% of subtotal)
9	regularDeliveryFee	float	0.0	Shipping fee for regular delivery items
10	rushDeliveryFee	float	0.0	Shipping fee for rush delivery items
11	totalAmount	float	0.0	Total amount including VAT and all delivery fees
12	deliveryInfo	DeliveryInfo	null	Delivery information associated with this order
13	paymentTransactionId	int	null	Reference to payment transaction
14	VAT_RATE	float	0.1	Value-added tax rate (constant 10%)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	Order()	void	Constructor for creating a new Order object
2	updateToReject()	void	Sets the order's status to "rejected"
3	updateToApprove()	void	Sets the order's status to "approved"
4	displayApproveStatus()	void	Shows or logs that the order has been approved
5	getOrderDetails	Order	Retrieves/shows the order's item list, price, etc.
6	updateStatus()	void	Updates this order's status based on input
7	save()	void	Persists this order in storage
8	calculateSubtotal	float	Calculates the sum of all product prices excluding VAT
9	calculateVAT	float	Calculates the VAT amount based on subtotal
10	calculateRegularDeliveryFee	float	Calculates shipping fee for regular delivery items
11	calculateRushDeliveryFee	float	Calculates shipping fee for rush delivery items
12	calculateTotalAmount	float	Calculates the total amount including VAT and all delivery fees

13	isEligibleForFreeShipping	boolean	Checks if order is eligible for free regular shipping
14	createOrderListScreen	float	Creates a screen listing multiple orders (for manager UI)
15	getOrderById	Order	Retrieves an order by its ID

Parameter:

- **updateStatus(newStatus: String): void**
  - **newStatus: T**he updated status ("pending", "approved", "rejected", "canceled", etc.).
- **getOrderById(orderId: int): Order**
  - **orderId:** The identifier of the order to retrieve.
  - **Return:** The Order object if found; otherwise might throw an exception.
- **calculateSubtotal(orderItems: List<OrderItem>): float**
  - **orderItems:** The list of ordered items to sum up.
  - **Return:** The subtotal of all items excluding VAT.
- **calculateVAT(subtotal: float): float**
  - subtotal: The total cost of items before VAT.
  - Return: subtotal * VAT_RATE.
- **isEligibleForFreeShipping(): boolean**
  - No parameters. Uses internal order data (subtotal, location, etc.).
  - Return: true if the order meets free shipping criteria, otherwise false.

Exception:

- **OrderNotFoundException**
  - Thrown if no order exists with the specified orderId when calling methods like getOrderById().
- **OrderAlreadyProcessedException**
  - Thrown if attempting to approve, reject, or cancel an order that has already been processed (i.e., not in "pending").

Method:

- **Order() (constructor)**
  - **Purpose:** Initializes a new order object with default values (status = "pending", createdDate = current time).
  - **Usage:** Called when the system or user creates an order (e.g., after finalizing a cart).
- **updateToReject()**
  - **Purpose:** Changes the order's status to "rejected".
  - **Usage:** Called by a manager or system logic if an order cannot be fulfilled (out of stock, other reasons).
- **updateToApprove()**
  - **Purpose:** Changes the order's status to "approved".
  - **Usage:** Invoked once the manager or system confirms items are in stock and the order is valid.
- **save()**
  - **Purpose:** Persists this order's current data (status, items, totalAmount, etc.) to a database or file.
  - **Usage:** Called whenever the order state is updated (e.g., after approving or rejecting).
- **calculateSubtotal(), calculateVAT(), calculateRegularDeliveryFee(), calculateRushDeliveryFee(), calculateTotalAmount()**
  - **Purpose:** Each method breaks down the cost calculation to keep the code organized.
  - **Usage:** Typically chained together after the user finalizes or updates the order's items or delivery method.
- **isEligibleForFreeShipping()**
  - Purpose: Checks rules (like "subtotal > 100,000 VND" or other promotional logic) to see if shipping can be waived.
  - Usage: Called during fee calculation to adjust regularDeliveryFee if free shipping applies.

State

- If status = "approved" or "rejected" → The order is considered processed and cannot be changed to another status without special rules.
- If orderItems changes → Recalculate subtotal, vatAmount, rushDeliveryFee, regularDeliveryFee, totalAmount.
- If free shipping is applicable → regularDeliveryFee might be set to 0 or partial discount.
- If paymentTransactionId is set → A corresponding payment has been recorded for this order.